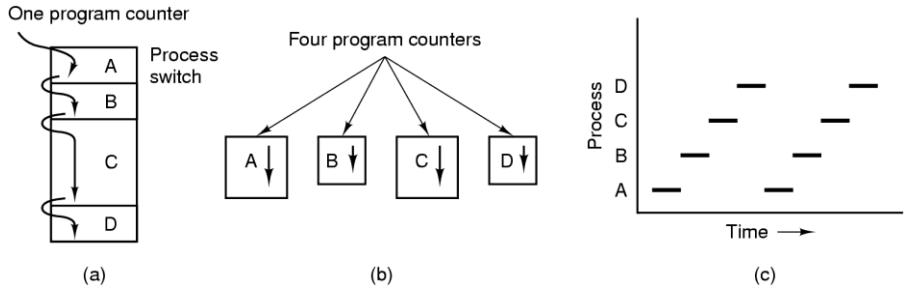
SISTEMAS OPERATIVOS

Aula Teórica 3

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

1. Processes
2. Threads
3. Event-Driven Servers
4. Synchronazation and Interprocess communication
5. Classical IPC problems
6. Scheduling



a) Multiprogramming of four programs

b) Conceptual model of 4 independent, sequential processes

c) Only one program active at any instant

Process Creation

Principal events that cause process creation

1- System initialization (ex. system boot; deamons (processes that stay in the background to handle some activity such as email, Web pages, news, printing, etc.))

2- Execution of a process-creation system call by a running process

(ex. if a large amount of data are being fetched over a network for subsequent processing, it may be convinient to create a process to fetch the data and put them in a shared buffer while a second process removes the data and processes them)

3- User request to create a new process

(ex. user can start a program by typing a command or double clicking on na icon -> open new window. These actions starts a new process and runs the program in it. Users may have multiple windows open at once, each running some process)

4- Initiation of a batch job (ex. batch systems found on large mainframes)

Process Termination

Conditions which terminate processes

1- Normal exit (voluntary) (ex. process have done their work; )

2- Error exit (voluntary) (ex. compiler error)

3- Fatal error (involuntary) (ex. program bug; dividing by zero)

4- Killed by another process (involuntary)

Parent creates a child process, and child processes can create its own process

Forms a hierarchy

In UNIX a process, and all its children and further descendents together form a "process group"

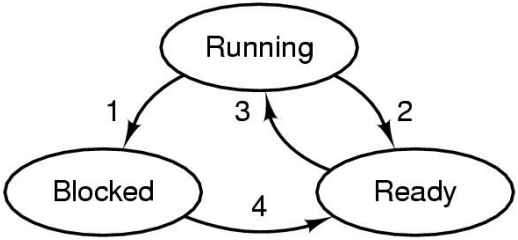
(ex. -When a users send a signal from the Keybord (e.g. CTRL+C), the signal is delivered to all members of the process group. All are killed by the signal.

- Just after computers boot , a special process callet *init*, starts running, it reads a file that tells how many terminals there are. Then it *forks* off a new process per terminal. These processes wait for someone to log in. If a log in is successful the login process executes a *shell* to accept commands. These comands may start up more processes, and so forth.

Thus all the processes in the whole system belong to a single tree, with *init* at the root.)

Windows has no concept of process hierarchy, all processes are created equal

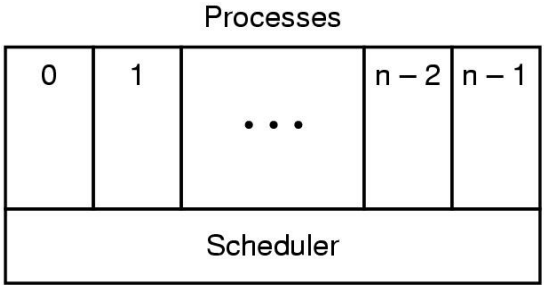
Process States (1)



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- Possible process states
  - 1- Running (actually using the CPU at that instant)
  - 2 - Ready (runnable; temporarily stopped to let another process run)
  - 3- Blocked (unable to run until some external event happens)
- Transitions between states shown

Process States (2)



- Lowest layer of process-structured OS
  - handles interrupts, scheduling
- Above that layer are sequential processes

Implementation of Processes (1)

Process management	Memory management	File management
Registers	Pointer to text segment	Root directory
Program counter	Pointer to data segment	Working directory
Program status word	Pointer to stack segment	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

- Some of the fields of a typical process-table entry

Implementation of Processes (2)

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly language procedure starts up new current process.

- Skeleton of what the lowest level of OS does when an interrupt occurs. The details may vary between operating systems.

Modeling Multiprogramming (1)

$$\text{CPU utilization} = 1 - p^n$$

Example:
-Computer has 8GB of memory
-The SO and its tables taking up 2 GB
-Each user program takes up 2GB
Just 3 users program are possible to be in memory at once, n=3.

With 80% I/O wait, we have a CPU utilization of $1 - 0.8^3$ or 49%.

Adding 8GB of memory to the computer we can have in memory n=7 user programs, raising the CPU utilization to 79%.
In other words, the additional 8GB will raise the throughput by 30%

Adding yet another 8 GB would increase the CPU utilization only from 79% to 91% thus raising the throughput by only another 12%.

Figure 2-6 shows, for different values of p (or "I/O wait"), the CPU utilization as a function of n, which is called the **degree of multiprogramming**.

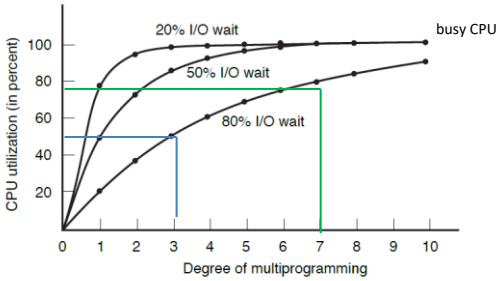


Figure 2-6. CPU utilization as a function of the number of processes in memory.

Using this model, the computer's owner might decide that the first memory addition was a good investment but the second was not.

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

